COMPUTER VISION

NAME: Harshavardhan. RV

REG NO: 192472163

COURSE CODE: ITA0515 - COMPUTER VISION

1. Perform basic Image Handling and processing operations on the image.

a. Read an image in python and Convert the given Image into Grayscale

INPUT:

```
from google.colab import files
from PIL import Image
import matplotlib.pyplot as plt
uploaded = files.upload()
filename = list(uploaded.keys())[0]
img = Image.open(filename)
gray_img = img.convert("L")
plt.figure(figsize=(10,4)) plt.subplot(1,2,1)
plt.title("Original Image")
plt.imshow(img)
plt.axis('off') plt.subplot(1,2,2)
plt.title("Grayscale Image")
plt.imshow(gray_img, cmap='gray')
plt.axis('off')
plt.show()
gray_img.save("grayscale_image.jpg")
print("Grayscale image saved as grayscale_image.jpg")
files.download("grayscale_image.jpg")
```

OUTPUT:

b. Read an image in python and Convert an Image to Blur using GaussianBlur.

INPUT:

```
from google.colab import files
from PIL import Image, ImageFilter
import matplotlib.pyplot as plt
uploaded = files.upload()
filename = list(uploaded.keys())[0]
img = Image.open(filename)
blur_img = img.filter(ImageFilter.GaussianBlur(radius=5))  # increase radius for more blur
plt.figure(figsize=(10,4))
plt.subplot(1,2,1)
plt.title("Original Image")
plt.imshow(img)
plt.axis("off")
plt.subplot(1,2,2)
plt.title("Gaussian Blurred Image")
plt.imshow(blur_img)
plt.axis("off")
plt.show()
blur_img.save("gaussian_blur_image.jpg")
print("Blurred image saved successfully!")
files.download("gaussian_blur_image.jpg")
```

OUTPUT:

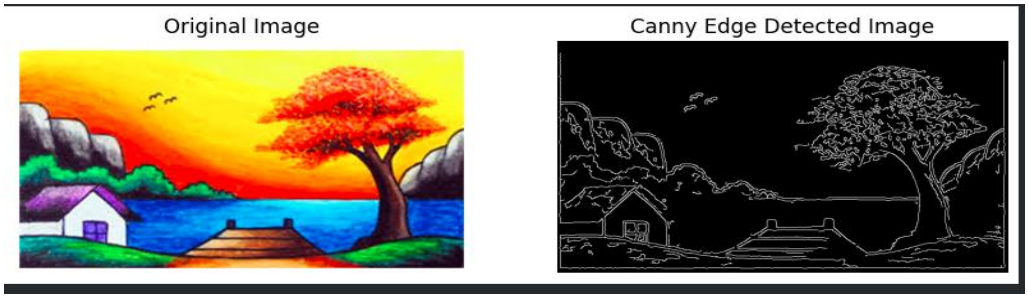
c. Read an image in python and Convert the given Image to show outline using Canny function.

INPUT:

```
from google.colab import files

import cv2

import matplotlib.pyplot as plt

import numpy as np

uploaded = files.upload()

filename = list(uploaded.keys())[0]

img = cv2.imread(filename)

img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

edges = cv2.Canny(gray, 100, 200)  # Adjust threshold for best results

plt.figure(figsize=(10,4))

plt.subplot(1,2,1)

plt.title("Original Image")

plt.imshow(img_rgb)

plt.axis('off')

plt.subplot(1,2,2)

plt.title("Canny Edge Detected Image")

plt.imshow(edges, cmap='gray')

plt.axis('off')

plt.show()
```

```
cv2.imwrite("canny_output.jpg", edges)
files.download("canny_output.jpg")
```

OUTPUT:



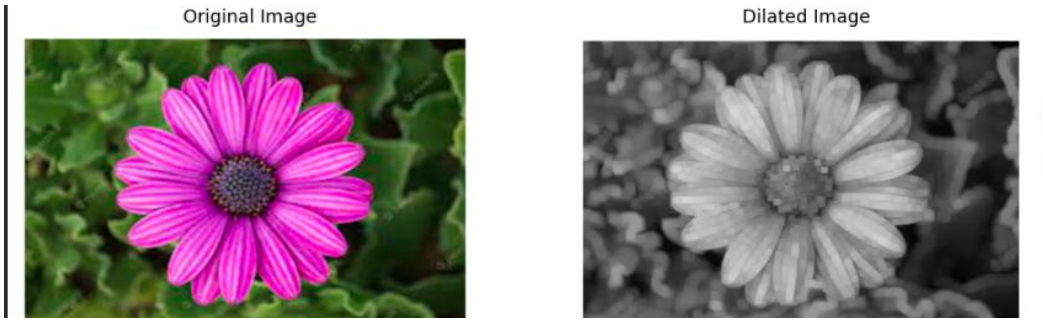
d. Read an image in python and Dilate an Image using Dilate function.

INPUT:

```
!pip install opencv-python-headless
import cv2
import numpy as np
from google.colab import files
import matplotlib.pyplot as plt
uploaded = files.upload()
image_path = next(iter(uploaded))
img = cv2.imread(image_path)
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
kernel = np.ones((7, 7), np.uint8)
dilated = cv2.dilate(gray, kernel, iterations=1)
plt.figure(figsize=(12, 6))
plt.subplot(1, 2, 1)
plt.title("Original Image")
plt.imshow(img_rgb)
plt.axis("off")
plt.subplot(1, 2, 2)
plt.title("Dilated Image")
plt.imshow(dilated, cmap="gray")
```

```
plt.axis("off")
plt.show()
```

OUTPUT:



e. Read an image in python and Erode an Image using erode function.

INPUT:

```
!pip install opencv-python-headless
import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files
uploaded = files.upload()
image_path = next(iter(uploaded))
img = cv2.imread(image_path)
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
kernel = np.ones((5, 5), np.uint8)   # 5×5 structuring element
eroded = cv2.erode(gray, kernel, iterations=1)
plt.figure(figsize=(12, 6))
plt.subplot(1, 2, 1)
plt.title("Original Image")
plt.imshow(img_rgb)
plt.axis("off")
plt.subplot(1, 2, 2)
plt.title("Eroded Image")
plt.imshow(eroded, cmap="gray")
```

```
plt.axis("off")
plt.show()
```

OUTPUT:

4.Scaling an image to its Bigger and Smaller sizes.

INPUT:

```
!pip install -q opencv-python-headless
import cv2
import numpy as np
from google.colab import files
import matplotlib.pyplot as plt
print("Upload an image file...")
uploaded = files.upload()
image_path = next(iter(uploaded.keys()))
img = cv2.imread(image_path)
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
bigger = cv2.resize(img, None, fx=2, fy=2, interpolation=cv2.INTER_LINEAR)
smaller = cv2.resize(img, None, fx=0.5, fy=0.5, interpolation=cv2.INTER_AREA)
plt.figure(figsize=(15, 5))
plt.subplot(1, 3, 1)
plt.imshow(img)
plt.title("Original Image")
plt.axis("off")
plt.subplot(1, 3, 2)
plt.imshow(bigger)
plt.title("Scaled Bigger (2x)")
plt.axis("off")
```

```
plt.subplot(1, 3, 3)
plt.imshow(smaller)
plt.title("Scaled Smaller (0.5x)")
plt.axis("off")
plt.show()
```

OUTPUT:

5. Perform Rotation of an image to clockwise and counter clockwise direction.

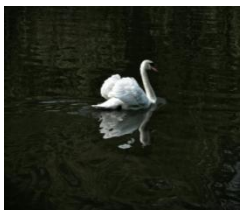
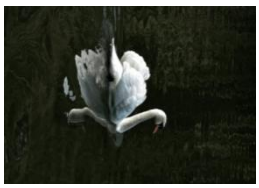
INPUT:

```
!pip install opencv-python
import cv2
import numpy as np
from google.colab.patches import cv2_imshow
from google.colab import files
uploaded = files.upload()
image_path = list(uploaded.keys())[0]
img = cv2.imread(image_path)
rows, cols = img.shape[:2]
angle_cw = -90
angle_ccw = 90
center = (cols // 2, rows // 2)
rotation_matrix_cw = cv2.getRotationMatrix2D(center, angle_cw, 1.0)
rotation_matrix_ccw = cv2.getRotationMatrix2D(center, angle_ccw, 1.0)
rotated_cw = cv2.warpAffine(img, rotation_matrix_cw, (cols, rows))
rotated_ccw = cv2.warpAffine(img, rotation_matrix_ccw, (cols, rows))
print("Original Image:")
cv2_imshow(img)
print("Clockwise Rotated Image:")
cv2_imshow(rotated_cw)
```

```
print("Counter-Clockwise Rotated Image:")
cv2_imshow(rotated_ccw)
```

OUTPUT:

ORIGINAL IMAGE | CLOCKWISE ROTATED | COUNTER CLOCK WISE ROTATED


6. Perform moving of an image from one place to another.

INPUT:

```
!pip install opencv-python
import cv2
import numpy as np
from google.colab.patches import cv2_imshow
from google.colab import files
uploaded = files.upload()
image_path = list(uploaded.keys())[0]
img = cv2.imread(image_path)
rows, cols = img.shape[:2]
tx = 100
ty = 50
translation_matrix = np.float32([[1, 0, tx],
                                 [0, 1, ty]])
moved_img = cv2.warpAffine(img, translation_matrix, (cols, rows))
print("Original Image:")
cv2_imshow(img)
print("Moved Image:")
cv2_imshow(moved_img)
```

OUTPUT:

ORIGINAL IMAGE MOVED IMAGE

:

7. Perform Affine Transformation on the image.

INPUT:

```
!pip install opencv-python
import cv2
import numpy as np
from google.colab.patches import cv2_imshow
from google.colab import files
uploaded = files.upload()
image_path = list(uploaded.keys())[0]
img = cv2.imread(image_path)
rows, cols, ch = img.shape
pts1 = np.float32([[50, 50], [200, 50], [50, 200]])
pts2 = np.float32([[70, 100], [220, 80], [100, 250]])
affine_matrix = cv2.getAffineTransform(pts1, pts2)
affine_output = cv2.warpAffine(img, affine_matrix, (cols, rows))
print("Original Image:")
cv2_imshow(img)
print("Affine Transformed Image:")
cv2_imshow(affine_output)
```

OUTPUT:

ORIGINAL IMAGE **AFFINE TRANSFORMED IMAGE**

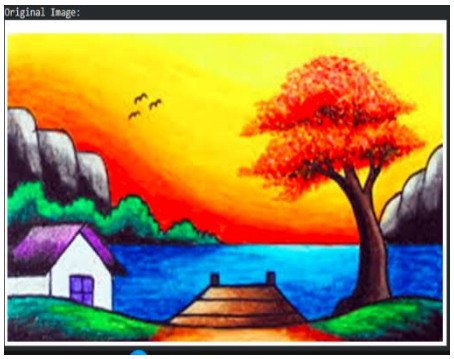
8. Perform Perspective Transformation on the image.

INPUT:

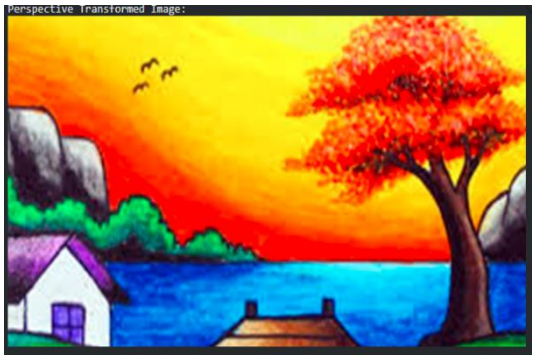
```
!pip install opencv-python
import cv2
import numpy as np
from google.colab.patches import cv2_imshow
from google.colab import files
uploaded = files.upload()
image_path = list(uploaded.keys())[0]
img = cv2.imread(image_path)
rows, cols, ch = img.shape
pts1 = np.float32([[50, 50], [cols - 50, 50], [50, rows - 50], [cols - 50, rows - 50]])
pts2 = np.float32([[0, 0], [cols - 1, 0], [0, rows - 1], [cols - 1, rows - 1]])
perspective_matrix = cv2.getPerspectiveTransform(pts1, pts2)
perspective_output = cv2.warpPerspective(img, perspective_matrix, (cols, rows))
print("Original Image:")
cv2_imshow(img)
print("Perspective Transformed Image:")
cv2_imshow(perspective_output)
```

OUTPUT:

ORIGINAL IMAGE



PERSPECTIVE TRANSFORMED IMAGE



10. Perform transformation using Homography matrix

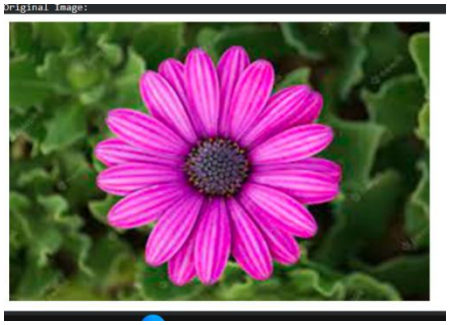
INPUT:

```
!pip install opencv-python
import cv2
import numpy as np
from google.colab.patches import cv2_imshow
from google.colab import files
uploaded = files.upload()
image_path = list(uploaded.keys())[0]
img = cv2.imread(image_path)
rows, cols, ch = img.shape
pts_src = np.float32([
    [50, 50],
    [cols - 50, 50],
    [50, rows - 50],
    [cols - 50, rows - 50]
])
pts_dst = np.float32([
    [0, 0],
    [cols - 1, 0],
```

```
    [100, rows - 1],
    [cols - 100, rows - 1]
])
H, status = cv2.findHomography(pts_src, pts_dst)
homography_output = cv2.warpPerspective(img, H, (cols, rows))
print("Original Image:")
cv2_imshow(img)
print("Homography Transformed Image:")
cv2_imshow(homography_output)
```

OUTPUT:

ORIGINAL IMAGE **HOMO TRANSFORMED IMAGE**



11. Perform transformation using Direct Linear Transformation.

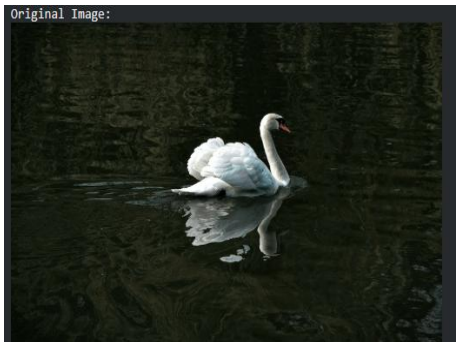
INPUT:

```
!pip install opencv-python numpy
import cv2
import numpy as np
from google.colab.patches import cv2_imshow
from google.colab import files
uploaded = files.upload()
image_path = list(uploaded.keys())[0]
img = cv2.imread(image_path)
rows, cols, ch = img.shape
pts_src = np.array([
    [50, 50],
```

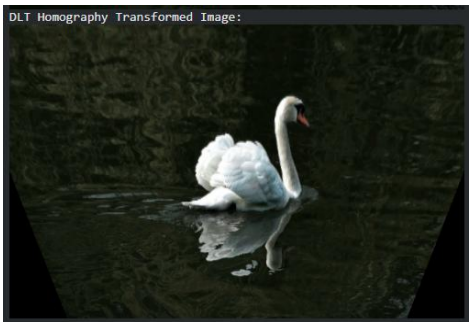
```
    [cols - 50, 50],
    [50, rows - 50],
    [cols - 50, rows - 50]
], dtype=np.float32)
pts_dst = np.array([
    [0, 0],
    [cols - 1, 0],
    [100, rows - 1],
    [cols - 100, rows - 1]
], dtype=np.float32)

def compute_homography(src_pts, dst_pts):
    A = []
    for i in range(len(src_pts)):
        x, y = src_pts[i][0], src_pts[i][1]
        u, v = dst_pts[i][0], dst_pts[i][1]
        A.append([-x, -y, -1, 0, 0, 0, x*u, y*u, u])
        A.append([0, 0, 0, -x, -y, -1, x*v, y*v, v])
    A = np.array(A)
    # Solve Ah = 0 using SVD
    U, S, V = np.linalg.svd(A)
    h = V[-1, :] / V[-1, -1]  # Normalize
    H = h.reshape(3, 3)
    return H
H_dlt = compute_homography(pts_src, pts_dst)
dlt_output = cv2.warpPerspective(img, H_dlt, (cols, rows))
print("Original Image:")
cv2_imshow(img)
print("DLT Homography Transformed Image:")
cv2_imshow(dlt_output)
```

OUTPUT:

ORIGINAL IMAGE



DIRECT LINEAR TRANSFORMED IMAGE



12. Perform Edge detection using canny method.

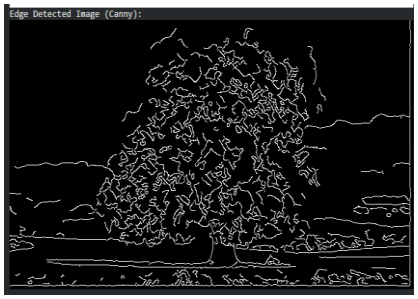
INPUT:

```
!pip install opencv-python
import cv2
import numpy as np
from google.colab.patches import cv2_imshow
from google.colab import files
uploaded = files.upload()
image_path = list(uploaded.keys())[0]
img = cv2.imread(image_path)
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
blurred = cv2.GaussianBlur(gray, (5, 5), 0)
edges = cv2.Canny(blurred, 100, 200)
print("Original Image:")
cv2_imshow(img)
print("Edge Detected Image (Canny):")
cv2_imshow(edges)
```

OUTPUT:

ORIGINAL IMAGE

EDGE DETECTED IMAGE



13. Perform Edge detection using Sobel Matrix along X axis.

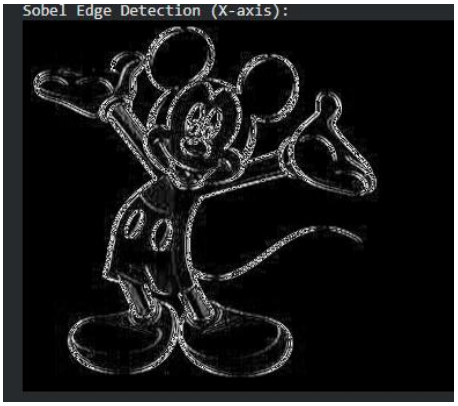
INPUT:

```
!pip install opencv-python
import cv2
import numpy as np
from google.colab.patches import cv2_imshow
from google.colab import files
uploaded = files.upload()
image_path = list(uploaded.keys())[0]
img = cv2.imread(image_path)
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
sobel_x = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
sobel_x = np.uint8(np.absolute(sobel_x))
print("Original Image:")
cv2_imshow(img)
print("Sobel Edge Detection (X-axis):")
cv2_imshow(sobel_x)
```

OUTPUT:

ORIGINAL IMAGE

SOBEL EDGE DETECTION (X-axis)



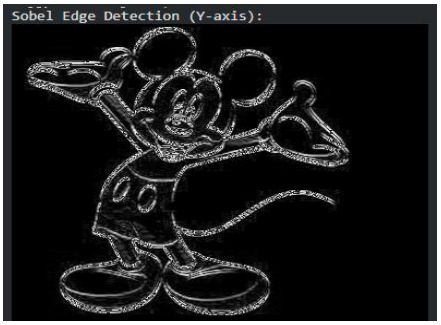
14. Perform Edge detection using Sobel Matrix along Y axis.

INPUT:

```
!pip install opencv-python
import cv2
import numpy as np
from google.colab.patches import cv2_imshow
from google.colab import files
uploaded = files.upload()
image_path = list(uploaded.keys())[0]
img = cv2.imread(image_path)
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
sobel_y = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)
sobel_y = np.uint8(np.absolute(sobel_y))
print("Original Image:")
cv2_imshow(img)
print("Sobel Edge Detection (Y-axis):")
cv2_imshow(sobel_y)
```

OUTPUT:

ORIGINAL IMAGE SOBEL EDGE DETECTION(Y-axis)



15. Perform Edge detection using Sobel Matrix along XY axis

```
!pip install opencv-python numpy matplotlib
import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files
uploaded = files.upload()
for fname in uploaded.keys():
    img_path = fname
img = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)
sobel_x = cv2.Sobel(img, cv2.CV_64F, 1, 0, ksize=3)
sobel_y = cv2.Sobel(img, cv2.CV_64F, 0, 1, ksize=3)
sobel_xy = cv2.magnitude(sobel_x, sobel_y)
plt.figure(figsize=(12,6))
plt.subplot(1,4,1)
plt.title("Original")
plt.imshow(img, cmap='gray')
plt.axis("off")
plt.subplot(1,4,2)
plt.title("Sobel X")
plt.imshow(np.abs(sobel_x), cmap='gray')
plt.axis("off")
plt.subplot(1,4,3)
plt.title("Sobel Y")
plt.imshow(np.abs(sobel_y), cmap='gray')
```

```
plt.axis("off")
plt.subplot(1,4,4)
plt.title("Sobel XY Magnitude")
plt.imshow(sobel_xy, cmap='gray')
plt.axis("off")
plt.tight_layout()
plt.show()
```

16. . Perform Sharpening of Image using Laplacian mask with negative center coefficient.

```
!pip install opencv-python numpy matplotlib
import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files
uploaded = files.upload()
for fname in uploaded.keys():
    img_path = fname
img = cv2.imread(img_path)
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
gray = cv2.cvtColor(img, cv2.COLOR_RGB2GRAY)
laplacian_kernel = np.array([[0, 1, 0],
                             [1, -4, 1],
                             [0, 1, 0]])
```

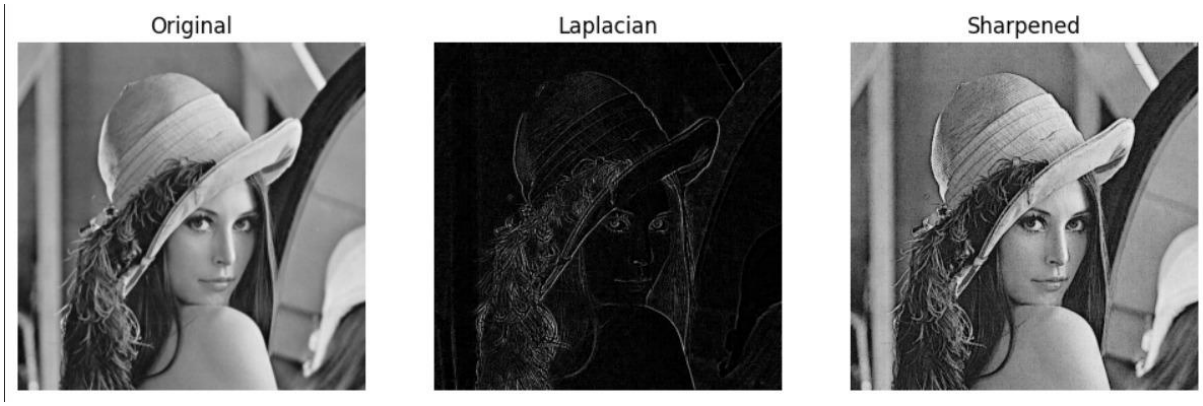
```
laplacian = cv2.filter2D(gray, cv2.CV_64F, laplacian_kernel)
sharpened = gray - laplacian
sharpened = np.clip(sharpened, 0, 255).astype(np.uint8)
plt.figure(figsize=(12,6))
plt.subplot(1,3,1)
plt.title("Original Grayscale")
plt.imshow(gray, cmap='gray')
plt.axis("off")
plt.subplot(1,3,2)
plt.title("Laplacian Response")
plt.imshow(np.abs(laplacian), cmap='gray')
plt.axis("off")
plt.subplot(1,3,3)
plt.title("Sharpened Image")
plt.imshow(sharpened, cmap='gray')
plt.axis("off")
plt.tight_layout()
plt.show()
```

17. Perform Sharpening of Image using Laplacian mask implemented with an extension of diagonal neighbors,

1	1	1
1	−8	1
1	1	1

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
!wget https://raw.githubusercontent.com/opencv/opencv/master/samples/data/lena.jpg -O sample.jpg
img = cv2.imread("sample.jpg")
img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
laplacian_mask = np.array([
    [1, 1, 1],
    [1, -8, 1],
    [1, 1, 1]
])
laplacian = cv2.filter2D(img, -1, laplacian_mask)
sharpened = cv2.subtract(img, laplacian)
plt.figure(figsize=(12,6))
plt.subplot(1,3,1)
plt.title("Original")
plt.imshow(img, cmap='gray')
plt.axis('off')
plt.subplot(1,3,2)
plt.title("Laplacian")
```

```
plt.imshow(laplacian, cmap='gray')
plt.axis('off')
plt.subplot(1,3,3)
plt.title("Sharpened")
plt.imshow(sharpened, cmap='gray')
plt.axis('off')
plt.show()
```



18. Perform Sharpening of Image using Laplacian mask with positive center coefficient.

Mask of Laplacian + addition

$$
\begin{aligned}
g(x,y) &= f(x,y) - [f(x+1,y) + f(x-1,y) \\
&\quad + f(x,y+1) + f(x,y-1) + 4f(x,y)] \\
&= 5f(x,y) - [f(x+1,y) + f(x-1,y) \\
&\quad + f(x,y+1) + f(x,y-1)]
\end{aligned}
$$

0	-1	0
-1	5	-1
0	-1	0

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
# Upload image
from google.colab import files
uploaded = files.upload()
# Read the image
for fn in uploaded.keys():
```

```
    img = cv2.imread(fn, cv2.IMREAD_GRAYSCALE)
# Laplacian mask with positive center coefficient
laplacian_mask = np.array([[0, -1, 0],
                           [-1, 5, -1],
                           [0, -1, 0]])

sharpened_img = cv2.filter2D(img, -1, laplacian_mask)
# Display results
plt.figure(figsize=(10,5))
plt.subplot(1,2,1)
plt.title("Original Image")
plt.imshow(img, cmap='gray')
plt.axis('off')
plt.subplot(1,2,2)
plt.title("Sharpened Image")
plt.imshow(sharpened_img, cmap='gray')
plt.axis('off')
plt.show()
```

20. Perform Sharpening of Image using High-Boost Masks.

High-boost Masks

0	−1	0	−1	−1	−1
−1	$A+4$	−1	−1	$A+8$	−1
0	−1	0	−1	−1	−1

- $A \geq 1$
- if A = 1, it becomes “standard” Laplacian sharpening

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

!wget https://raw.githubusercontent.com/opencv/opencv/master/samples/data/lena.jpg -O sample.jpg

img = cv2.imread("sample.jpg")
img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

A = 2

highboost_mask = np.array([
    [0, -1, 0],
    [-1, A+4, -1],
    [0, -1, 0]
])

filtered = cv2.filter2D(img, -1, highboost_mask)

plt.figure(figsize=(12,6))

plt.subplot(1,3,1)
plt.title("Original")
plt.imshow(img, cmap='gray')
plt.axis('off')

plt.subplot(1,3,2)
```

```
plt.title(f"High-Boost Mask (A={A})")
plt.imshow(filtered, cmap='gray')
plt.axis('off')

plt.subplot(1,3,3)
plt.title("Sharpened Output")
plt.imshow(filtered, cmap='gray')
plt.axis('off')

plt.show()
```

19. Perform Sharpening of Image using unsharp masking.

Unsharp masking

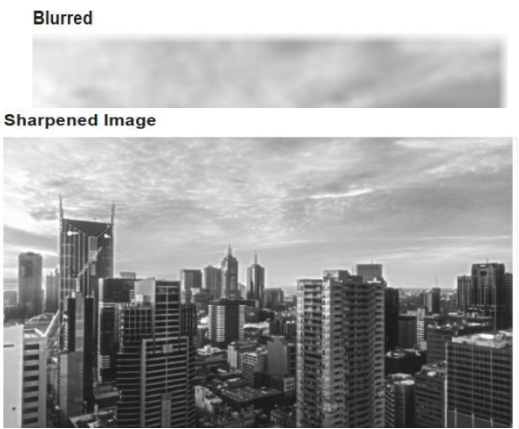
$$f_s(x,y) = f(x,y) - \bar{f}(x,y)$$

sharpened image = original image – blurred image

- to subtract a blurred version of an image produces sharpening output image.


```
import cv2
import numpy as np
import IPython.display as display
import base64
```

```
fn = 'Screenshot 2025-12-05 154454.png'

if 'uploaded' in globals() and fn in uploaded:
    nparr = np.frombuffer(uploaded[fn], np.uint8)
    img = cv2.imdecode(nparr, cv2.IMREAD_COLOR)

    if img is None:
        print(f"Error: cv2.imdecode failed for {fn}.")
    else:
        gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
        blur = cv2.GaussianBlur(gray, (9, 9), 10)
        unsharp_mask = cv2.subtract(gray, blur)
        sharpened = cv2.add(gray, unsharp_mask)

        def display_cv_image(img_data, title=""):
            _, buffer = cv2.imencode('.png', img_data)
            img_str = base64.b64encode(buffer).decode()
            return display.HTML(f'<h3>{title}</h3><img src="{img_str}" />')

        display.display(display_cv_image(gray, "Original (Grayscale)"))
        display.display(display_cv_image(blur, "Blurred"))
        display.display(display_cv_image(unsharp_mask, "Unsharp Mask"))
        display.display(display_cv_image(sharpened, "Sharpened Image"))
else:
    print(f"Error: Image file '{fn}' not found.")
```



21. Perform Sharpening of Image using Gradient masking.

−1	−2	−1
0	0	0
1	2	1

−1	0	1
−2	0	2
−1	0	1

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread("/content/Screenshot 2025-12-05 155031.png")

if img is None:
    print("Error: Could not load image. Check path.")
else:
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    mask1 = np.array([[-1, -2, -1],
                      [ 0,  0,  0],
                      [ 1,  2,  1]])

    mask2 = np.array([[-1,  0,  1],
                      [-2,  0,  2],
                      [-1,  0,  1]])

    grad1 = cv2.filter2D(gray, -1, mask1)
    grad2 = cv2.filter2D(gray, -1, mask2)
    gradient = cv2.addWeighted(grad1, 0.5, grad2, 0.5, 0)
    sharpened = cv2.add(gray, gradient)
```

```
    images = [gray, grad1, grad2, gradient, sharpened]
    titles = ["Original", "Mask 1 Output", "Mask 2 Output", "Combined Gradient", "Sharpened"]

    plt.figure(figsize=(15, 4))

    for i in range(5):
        plt.subplot(1, 5, i+1)
        plt.imshow(images[i], cmap='gray')
        plt.title(titles[i], fontsize=8)
        plt.axis('off')

    plt.tight_layout()
plt.show()
```

22. Insert water marking to the image using OpenCV.

```
import cv2
import numpy as np
from google.colab.patches import cv2_imshow

image = cv2.imread(list(uploaded.keys())[0])
rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

watermark_text = "WATERMARK"
x, y = 30, 50
font = cv2.FONT_HERSHEY_SIMPLEX
```

```
font_scale = 1.2

color = (255, 255, 255)

thickness = 2


watermarked_img = rgb.copy()

cv2.putText(watermarked_img, watermark_text, (x, y), font, font_scale, color, thickness,
cv2.LINE_AA)


cv2_imshow(watermarked_img)


cv2.imwrite("watermarked_image.jpg", cv2.cvtColor(watermarked_img, cv2.COLOR_RGB2BGR))


print("Watermarked image saved as watermarked_image.jpg")
```

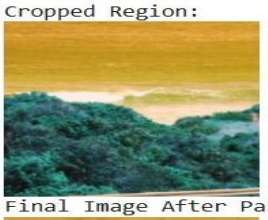

```
Watermarked image saved as watermarked_image.jpg
```

23. Find the boundary of the image using Convolution kernel for the given image.

```
import cv2

from google.colab.patches import cv2_imshow


img1 = cv2.imread("/content/Screenshot 2025-12-05 161545.png")

img2 = cv2.imread("/content/Screenshot 2025-12-05 161630.png")


if img1 is None or img2 is None:

    print("Error: Could not load one or both images. Check file paths.")

else:
```

```
img1_rgb = cv2.cvtColor(img1, cv2.COLOR_BGR2RGB)
img2_rgb = cv2.cvtColor(img2, cv2.COLOR_BGR2RGB)

y1, y2 = 50, 200
x1, x2 = 50, 200
crop = img1_rgb[y1:y2, x1:x2]

print("Cropped Region:")
cv2_imshow(crop)

h, w, _ = crop.shape
H, W, _ = img2_rgb.shape
if h > H or w > W:
    print("Cropped region too large. Resizing to fit target image.")
    crop = cv2.resize(crop, (W//3, H//3))
img2_result = img2_rgb.copy()
img2_result[0:crop.shape[0], 0:crop.shape[1]] = crop
print("Final Image After Pasting:")
cv2_imshow(img2_result)
cv2.imwrite("final_pasted_image.jpg", cv2.cvtColor(img2_result, cv2.COLOR_RGB2BGR))
print("Saved as final_pasted_image.jpg")
```

```
Cropped Region:
```



```
Final Image After Pasting:
```

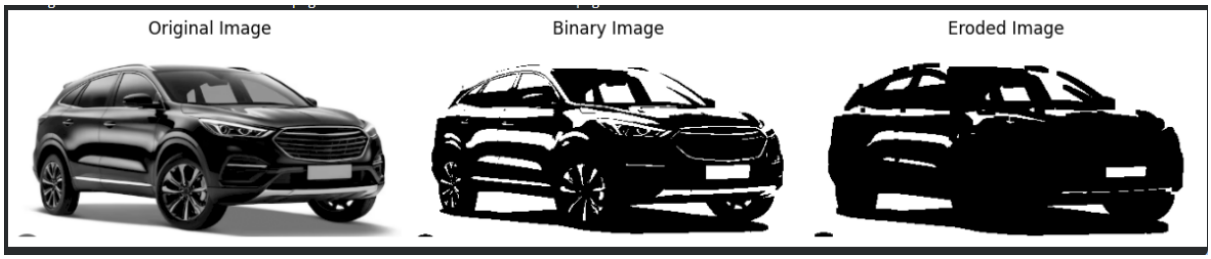
```
Saved as final_pasted_image.jpg
```

24. Morphological operations based on OpenCV using Erosion technique.

INPUT:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files

# --- Upload Image ---
uploaded = files.upload()
filename = list(uploaded.keys())[0]

# --- Read Image ---
img = cv2.imread(filename, cv2.IMREAD_GRAYSCALE)

# --- Threshold to get a Binary Image ---
_, binary = cv2.threshold(img, 127, 255, cv2.THRESH_BINARY)

# --- Create Structuring Element (Kernel) ---
kernel = np.ones((5,5), np.uint8)

# --- Erosion Operation ---
eroded = cv2.erode(binary, kernel, iterations=1)

# --- Display Results Side-by-Side ---
plt.figure(figsize=(12,4))

plt.subplot(1,3,1)
plt.imshow(img, cmap='gray')
plt.title("Original Image")
plt.axis("off")

plt.subplot(1,3,2)
plt.imshow(binary, cmap='gray')
plt.title("Binary Image")
plt.axis("off")

plt.subplot(1,3,3)
plt.imshow(eroded, cmap='gray')
plt.title("Eroded Image")
plt.axis("off")

plt.tight_layout()
plt.show()
```

OUTPUT:



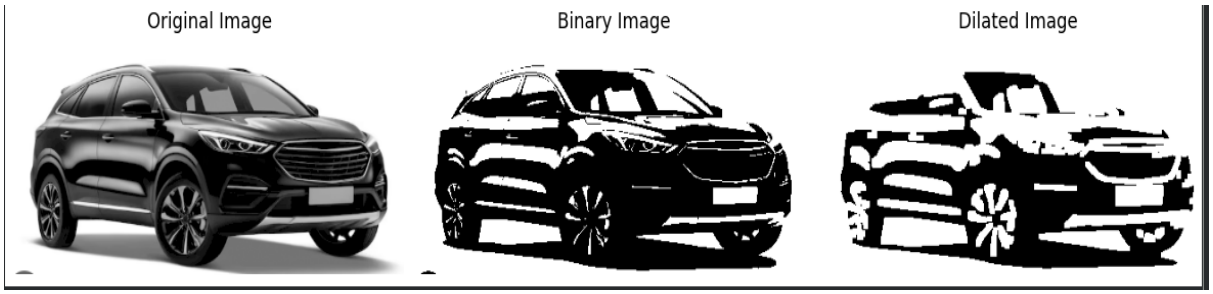
25. Morphological operations based on OpenCV using Dilation technique.

INPUT:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files

# --- Upload Image ---
uploaded = files.upload()
filename = list(uploaded.keys())[0]

# --- Read Image ---
img = cv2.imread(filename, cv2.IMREAD_GRAYSCALE)

# --- Convert to Binary Image ---
_, binary = cv2.threshold(img, 127, 255, cv2.THRESH_BINARY)

# --- Create Structuring Element (Kernel) ---
kernel = np.ones((5,5), np.uint8)

# --- Dilation Operation ---
dilated = cv2.dilate(binary, kernel, iterations=1)

# --- Display Results Side-by-Side ---
plt.figure(figsize=(12,4))

plt.subplot(1,3,1)
plt.imshow(img, cmap='gray')
plt.title("Original Image")
plt.axis("off")

plt.subplot(1,3,2)
plt.imshow(binary, cmap='gray')
plt.title("Binary Image")
plt.axis("off")

plt.subplot(1,3,3)
plt.imshow(dilated, cmap='gray')
plt.title("Dilated Image")
```

```
plt.axis("off")
```

OUTPUT:



26. Morphological operations based on OpenCV using Opening technique

INPUT:

```
# -------------------------
# Morphological Opening using OpenCV
# -------------------------

import cv2
import matplotlib.pyplot as plt

# Load image
img = cv2.imread('/content/sample.jpg', 0)  # change path if needed

# Create structuring element (kernel)
kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (5,5))

# Apply Opening (erosion followed by dilation)
opening = cv2.morphologyEx(img, cv2.MORPH_OPEN, kernel)

# Display results side by side
plt.figure(figsize=(10,5))

plt.subplot(1,2,1)
plt.title("Original Image")
plt.imshow(img, cmap='gray')
```

```
plt.axis('off')


plt.subplot(1,2,2)
plt.title("After Opening")
plt.imshow(opening, cmap='gray')
plt.axis('off')


plt.tight_layout()
plt.show()
```

OUTPUT:

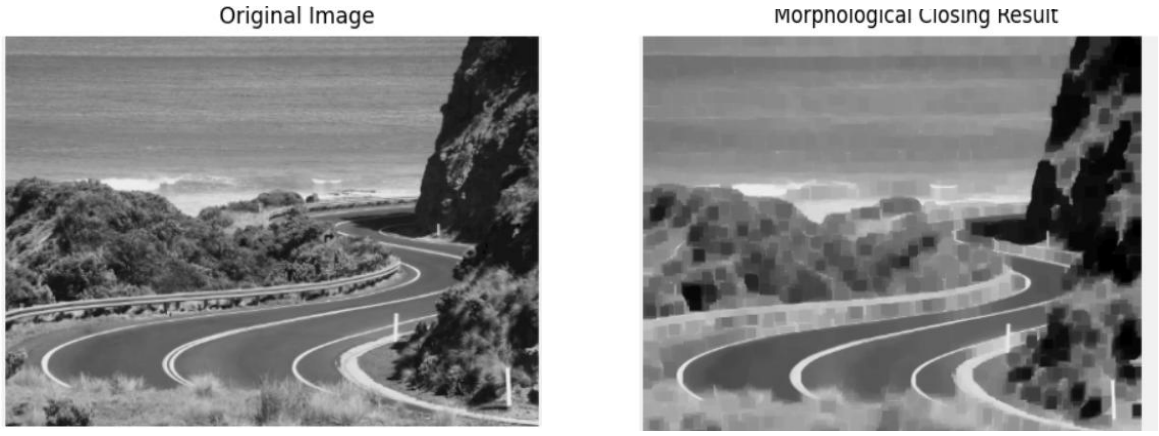
27. Morphological operations based on OpenCV using Closing technique.

INPUT:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files
uploaded = files.upload()
for filename in uploaded.keys():
    img = cv2.imdecode(np.frombuffer(uploaded[filename], np.uint8), cv2.IMREAD_GRAYSCALE)

# Display original image
plt.figure(figsize=(6,6))
plt.title("Original Image")
plt.imshow(img, cmap='gray')
plt.axis('off')
```

```
plt.show()
# Create structuring element
kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (7, 7))  # 7x7 kernel

# Apply Closing operation
closing = cv2.morphologyEx(img, cv2.MORPH_CLOSE, kernel)

# Display result
plt.figure(figsize=(6,6))
plt.title("Morphological Closing Result")
plt.imshow(closing, cmap='gray')
plt.axis('off')
plt.show()
```

OUTPUT:



28. Morphological operations based on OpenCV using Morphological Gradient technique.

INPUT:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files
uploaded = files.upload()
for filename in uploaded.keys():
    img = cv2.imdecode(np.frombuffer(uploaded[filename], np.uint8), cv2.IMREAD_GRAYSCALE)
```

```
# Create structuring element (kernel)
kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (5, 5))

# Apply Morphological Gradient
gradient = cv2.morphologyEx(img, cv2.MORPH_GRADIENT, kernel)

# Display Original and Gradient Result side-by-side
plt.figure(figsize=(12,6))

plt.subplot(1,2,1)
plt.title("Original Image")
plt.imshow(img, cmap='gray')
plt.axis('off')

plt.subplot(1,2,2)
plt.title("Morphological Gradient Result")
plt.imshow(gradient, cmap='gray')
plt.axis('off')

plt.show()
```

OUTPUT:

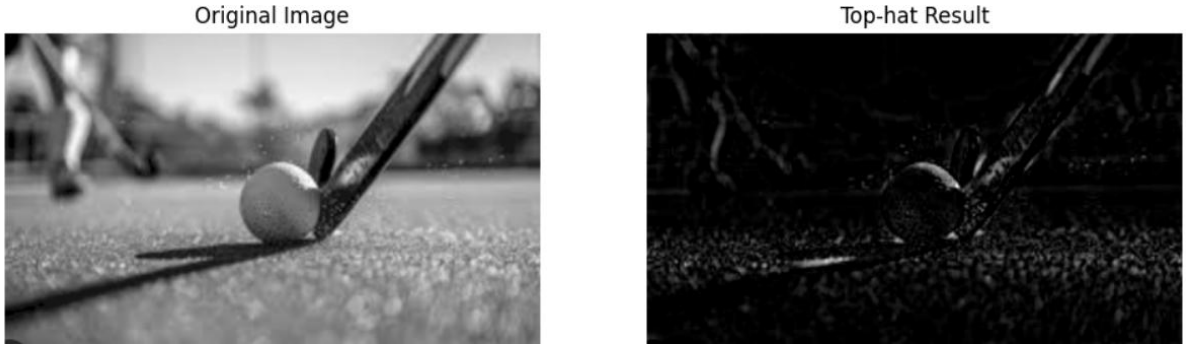
29. Morphological operations based on OpenCV using Top hat technique.

INPUT:

```
# ============================================================
# Morphological Top-hat using OpenCV
# ============================================================
import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files

# Upload an image
uploaded = files.upload()

# Load image in grayscale
for filename in uploaded.keys():
    img = cv2.imdecode(np.frombuffer(uploaded[filename], np.uint8), cv2.IMREAD_GRAYSCALE)

# Create structuring element (kernel)
kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (11, 11))  # You can change size

# Apply Top-hat operation
tophat = cv2.morphologyEx(img, cv2.MORPH_TOPHAT, kernel)

# Display Original and Top-hat Result side-by-side
plt.figure(figsize=(12, 6))

plt.subplot(1, 2, 1)
plt.title("Original Image")
plt.imshow(img, cmap='gray')
plt.axis('off')

plt.subplot(1, 2, 2)
plt.title("Top-hat Result")
```

```
plt.imshow(tophat, cmap='gray')
plt.axis('off')

plt.show()
```

OUTPUT:



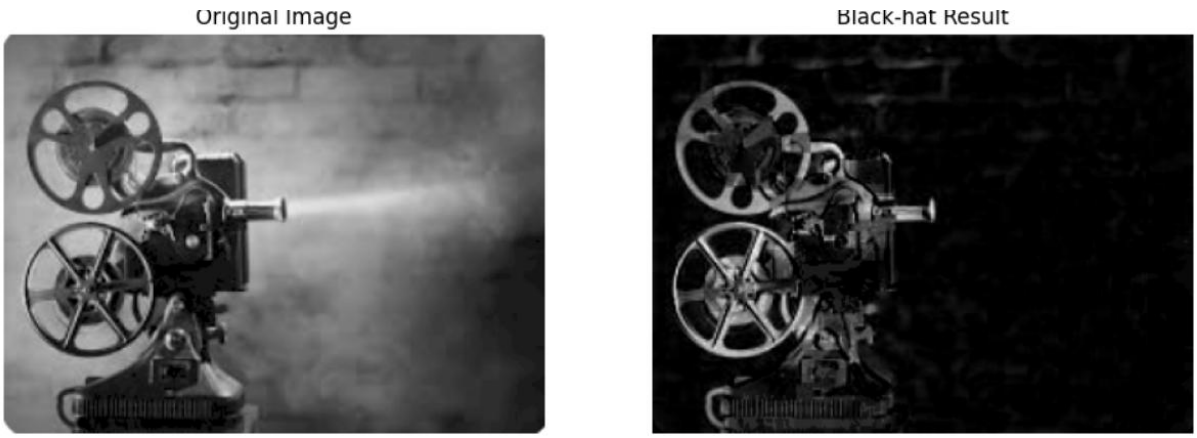
30. Morphological operations based on OpenCV using Black hat technique.

INPUT:

```
# ============================================================
# Morphological Black-hat using OpenCV
# ============================================================
import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files

# Upload an image
uploaded = files.upload()

# Load image in grayscale
for filename in uploaded.keys():
    img = cv2.imdecode(np.frombuffer(uploaded[filename], np.uint8), cv2.IMREAD_GRAYSCALE)

# Create structuring element (kernel)
kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (11, 11))  # adjustable size
```

```
# Apply Black-hat operation
blackhat = cv2.morphologyEx(img, cv2.MORPH_BLACKHAT, kernel)

# Display Original and Black-hat Result side-by-side
plt.figure(figsize=(12, 6))

plt.subplot(1, 2, 1)
plt.title("Original Image")
plt.imshow(img, cmap='gray')
plt.axis('off')

plt.subplot(1, 2, 2)
plt.title("Black-hat Result")
plt.imshow(blackhat, cmap='gray')
plt.axis('off')

plt.show()
```

OUTPUT:



31. Recognise watch from the given image by general Object recognition using OpenCV.

INPUT:

32. # ----------

PLAY VIDEO IN REVERSE USING OPENCV (GOOGLE COLAB)

```
import cv2
from google.colab import files
import matplotlib.pyplot as plt

# ---------- STEP 1: Upload Video ----------
print("Please upload your video file (mp4 / avi / mov):")
uploaded = files.upload()

video_path = list(uploaded.keys())[0]  # Get uploaded file name

# ---------- STEP 2: Read all frames ----------
cap = cv2.VideoCapture(video_path)

frames = []
while True:
    ret, frame = cap.read()
    if not ret:
        break
```

```
    frames.append(frame)

cap.release()
print("Total frames read:", len(frames))

# ---------- STEP 3: Display frames in reverse -------
for frame in reversed(frames):
    frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    plt.imshow(frame_rgb)
    plt.axis('off')
    plt.show()
```

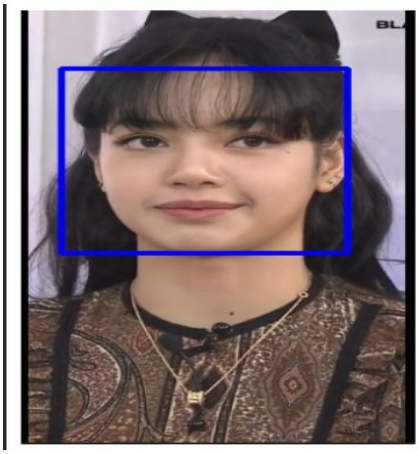
output : video executed

33. Face Detection using Opencv

```
# ------------------------------------------------------------
# INSTALL OPENCV
# ------------------------------------------------------------
!pip install opencv-python opencv-python-headless

import cv2
import matplotlib.pyplot as plt
import numpy as np
from google.colab import files

# ------------------------------------------------------------
# 1. Upload an image (face image)
# ------------------------------------------------------------
uploaded = files.upload()
image_path = next(iter(uploaded))   # get the uploaded image name

# ------------------------------------------------------------
```

```
# 2. Load image
# ---------------------------------------------------------
img = cv2.imread(image_path)
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# ---------------------------------------------------------
# 3. Load Haar Cascade Face Detector
# ---------------------------------------------------------
cascade_url =
"https://raw.githubusercontent.com/opencv/opencv/master/data/haarcascades/haarcascade_frontalface_default.xml"
!wget -q $cascade_url -O haarcascade_frontalface_default.xml

face_cascade = cv2.CascadeClassifier("haarcascade_frontalface_default.xml")

# ---------------------------------------------------------
# 4. Detect faces
# ---------------------------------------------------------
faces = face_cascade.detectMultiScale(gray, scaleFactor=1.1, minNeighbors=4)

# Draw bounding boxes
for (x, y, w, h) in faces:
    cv2.rectangle(img, (x, y), (x+w, y+h), (255, 0, 0), 3)

# ---------------------------------------------------------
# 5. Display output
# ---------------------------------------------------------
plt.figure(figsize=(6,6))
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.axis("off")
plt.show()
```

print("Faces detected:", len(faces))

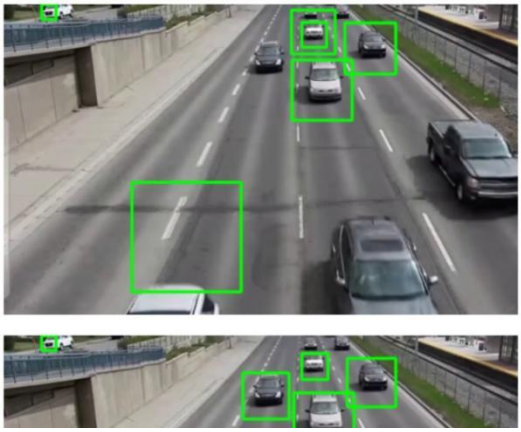


34. 34. Vehicle Detection in a Video frame using OpenCV .

```
# ------------------------------------------------------------
# Install OpenCV
# ------------------------------------------------------------
!pip install opencv-python opencv-python-headless

import cv2
import matplotlib.pyplot as plt
from google.colab import files
import numpy as np

# ------------------------------------------------------------
# 1. Upload a video file
# ------------------------------------------------------------
uploaded = files.upload()
video_path = next(iter(uploaded))

# ------------------------------------------------------------
# 2. Download Haar Cascade for Vehicle Detection
# ------------------------------------------------------------
```

```
!wget -q
https://raw.githubusercontent.com/andrewssobral/vehicle_detection_haarcascades/master/cars.xml
-O cars.xml


car_cascade = cv2.CascadeClassifier("cars.xml")


# ---------------------------------------------------------

# 3. Open the uploaded video

# ---------------------------------------------------------

cap = cv2.VideoCapture(video_path)


print("Vehicle Detection Started...\n")


# ---------------------------------------------------------

# 4. Read frames one by one

# ---------------------------------------------------------

while True:

    ret, frame = cap.read()

    if not ret:

        break


    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)


    # Detect vehicles

    cars = car_cascade.detectMultiScale(gray, 1.2, 2)


    # Draw bounding boxes

    for (x, y, w, h) in cars:

        cv2.rectangle(frame, (x, y), (x + w, y + h), (0, 255, 0), 3)


    # Display the frame

    frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
```

```
    plt.imshow(frame_rgb)
    plt.axis("off")
    plt.show()

cap.release()
print("Vehicle Detection Completed.")
```

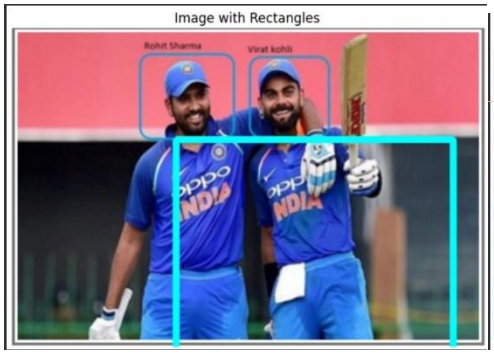


35. Draw Rectangular shape and extract objects.


```
# ----------------------------------------------------------
# Install OpenCV
# ----------------------------------------------------------
!pip install opencv-python opencv-python-headless

import cv2
import numpy as np
import matplotlib.pyplot as plt
```

```
from google.colab import files

# ---------------------------------------------------------
# 1. Upload your image
# ---------------------------------------------------------
uploaded = files.upload()
image_path = next(iter(uploaded))

# Load image
img = cv2.imread(image_path)
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
output = img_rgb.copy()

# ---------------------------------------------------------
# 2. MANUALLY DEFINE RECTANGLE COORDINATES
# (you can adjust these values to fit your image)
# ---------------------------------------------------------

# Example rectangles (x, y, width, height)
rectangles = [
    (120, 80, 200, 260),   # Object 1
    (350, 80, 200, 260)    # Object 2
]

# ---------------------------------------------------------
# 3. Draw rectangles AND extract objects
# ---------------------------------------------------------
extracted_objects = []   # to store cropped images

for (x, y, w, h) in rectangles:
    # Draw rectangle
```

```
    cv2.rectangle(output, (x, y), (x + w, y + h), (0, 255, 255), 4)

    # Extract object
    crop = img_rgb[y:y+h, x:x+w]
    extracted_objects.append(crop)

# ----------------------------------------------------------
# 4. Display the image with rectangles
# ----------------------------------------------------------
plt.figure(figsize=(8,6))
plt.title("Image with Rectangles")
plt.imshow(output)
plt.axis("off")
plt.show()

# ----------------------------------------------------------
# 5. Display extracted objects
# ----------------------------------------------------------
for i, obj in enumerate(extracted_objects):
    plt.figure(figsize=(4,4))
    plt.title(f"Extracted Object {i+1}")
    plt.imshow(obj)
    plt.axis("off")
    plt.show()
```



22A]

```
# ------------------------------------------------------------
# FIND THE BOUNDARY OF AN IMAGE USING CONVOLUTION KERNEL
# ------------------------------------------------------------

import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files

# ------------------------------
# 1. Upload image
# ------------------------------
uploaded = files.upload()

image_path = list(uploaded.keys())[0]

# Read image
img = cv2.imread(image_path, 0)  # grayscale
plt.figure(figsize=(6,6))
plt.title("Original Image")
plt.imshow(img, cmap='gray')
plt.axis('off')
plt.show()

# ------------------------------------------------
# 2. Convolution Kernel for Boundary Detection
#    (Laplacian Kernel)
# ------------------------------------------------
kernel = np.array([[ -1, -1, -1],
                   [ -1,  8, -1],
                   [ -1, -1, -1]])
```

```
# Apply convolution manually
boundary = cv2.filter2D(img, -1, kernel)

plt.figure(figsize=(6,6))
plt.title("Boundary Image using Convolution Kernel")
plt.imshow(boundary, cmap='gray')
plt.axis('off')
plt.show()
```